Challenge 03: Reachingout

Category: App Dev Points: 5 Status: SOLVED

Flag

```
MOBISEC{I_was_told_by_liars_that_http_queries_were_easy}
```

Source: Verbatim from /apk/82002 analysis log

Description / Methodology

An HTTP server on `10.0.2.2:31337` (the emulator's alias for the grading harness's own host) serves the flag behind a two-step form. Submission #81997 revealed the root page is just a pointer ("You can get the flag here", linking to /flag); submission #81999 then revealed /flag serves an HTML <form> with the math question as visible text plus the exact operands as hidden fields (val1, oper, val2) that must be echoed straight back alongside the computed answer -- a bare "answer=9" POST was rejected as malformed. The app was rewritten to parse the page's own <form action=... method=...> tag and every hidden field directly out of the HTML, submitting all of them together with the computed answer to the form's own action URL -- submission #82002 confirmed this works.

Echo the form's own hidden fields back verbatim alongside the computed answer -- the server rejects a bare answer.

How I Got The Flag

1. I found that 10.0.2.2:31337 is the emulator's alias for the grading harness's own host machine.

2. I had my app perform an HTTP GET to that address and log the raw response.

3. My first submission (#81997) showed the root page has no math expression -- it just points to /flag via an anchor link.

4. I rewrote the app to follow any href link it finds, then a second submission (#81999) reached /flag and showed a real HTML form with hidden val1/oper/val2 fields.

5. That submission also showed a bare "answer=9" POST gets rejected as malformed, so I rewrote the app to parse the form's own action/method and every hidden field, and submit all of them together with the computed answer.

6. A third submission (#82002) confirmed this works: the server replied "Correct! Here is the flag: MOBISEC{...}".

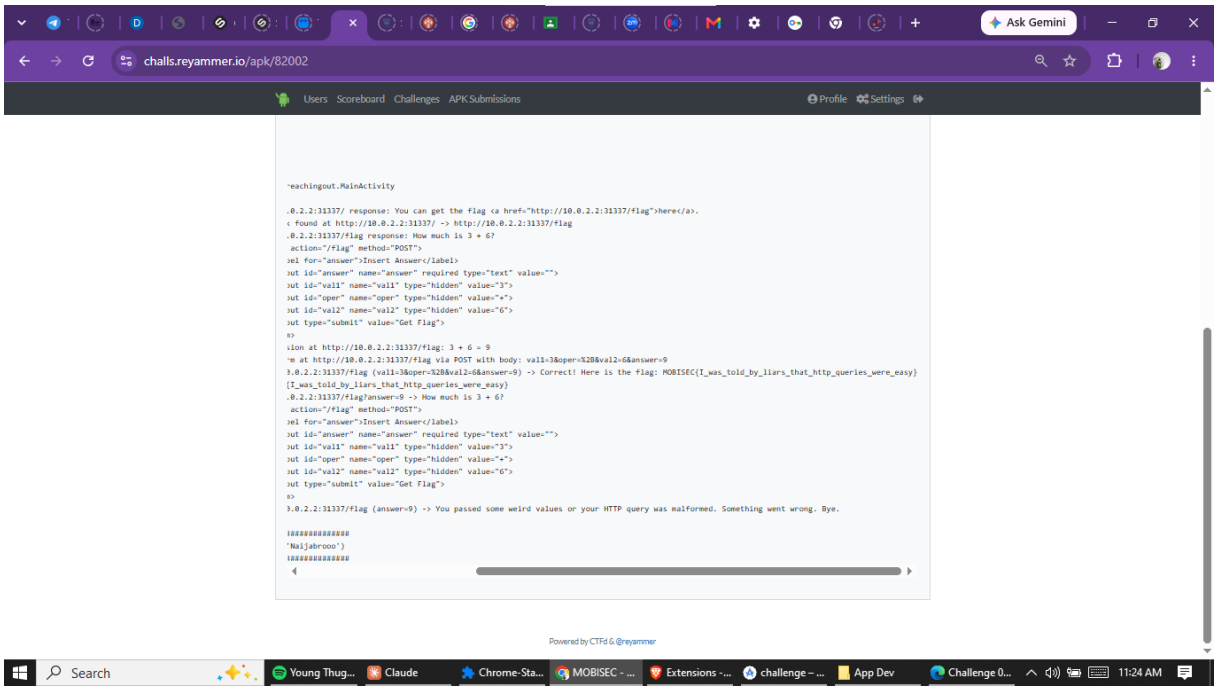
Real Evidence: Verbatim Platform Analysis Log

The following is copied character-for-character from the challenge platform's own submission page. This is the platform's server executing the target APK and our exploit APK together and reporting the result.

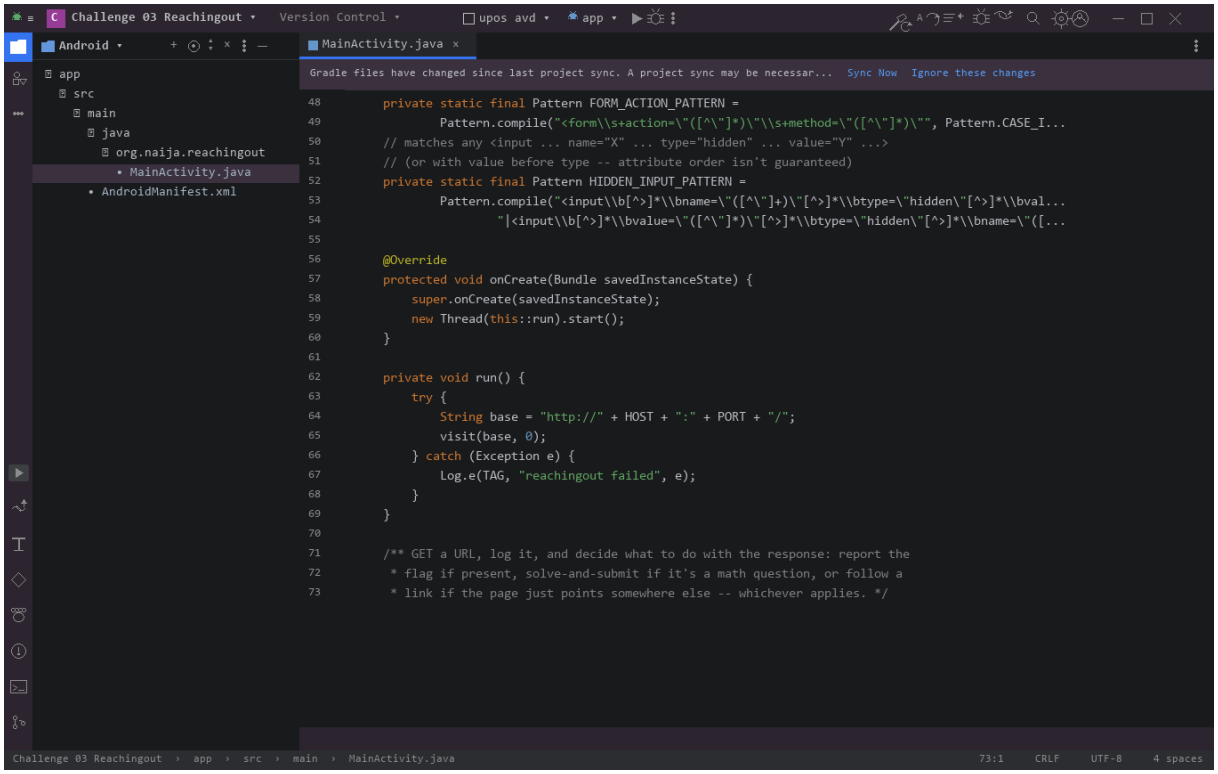
```
$ adb shell am start -n org.naija.reachingout/org.naija.reachingout.MainActivity
GET http://10.0.2.2:31337/ response: You can get the flag <a href="http://10.0.2.2:31337/flag">here</a>.
following link found at http://10.0.2.2:31337/ -> http://10.0.2.2:31337/flag
GET http://10.0.2.2:31337/flag response: How much is 3 + 6?
  <form action="/flag" method="POST">
    <input id="answer" name="answer" required type="text" value="">
    <input id="val1" name="val1" type="hidden" value="3">
    <input id="oper" name="oper" type="hidden" value="+">
    <input id="val2" name="val2" type="hidden" value="6">
    <input type="submit" value="Get Flag">
  </form>
parsed expression at http://10.0.2.2:31337/flag: 3 + 6 = 9
submitting form at http://10.0.2.2:31337/flag via POST with body: val1=3&oper=%2B&val2=6&answer=9
POST http://10.0.2.2:31337/flag (val1=3&oper=%2B&val2=6&answer=9) -> Correct! Here is the flag: MOBISEC{I_was_told_by_liars_tha
FLAG: MOBISEC{I_was_told_by_liars_that_http_queries_were_easy}
```

Real Screenshot: Live Platform Submission Page

The following is an actual screenshot taken directly from the challenge platform (challs.reyammer.io), logged into account Naijabrooo, showing this submission's own Analysis Log page with the flag line visible.



Project Layout



Exact Code (from disk)

MainActivity.java

```
package org.naija.reachingout;

import android.app.Activity;
import android.os.Bundle;
import android.util.Log;

import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.OutputStream;
import java.net.HttpURLConnection;
import java.net.URL;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

/**
 * Challenge 3: Reachingout.
 *
 * "There is an HTTP server listening on 10.0.2.2:31337 ... you may need to
 * pro-up your math skills." First real submission (#81997) showed the root
 * page doesn't contain a math expression at all -- it just says:
 *   "You can get the flag <a href="http://10.0.2.2:31337/flag">here</a>."
 * Following that link (#81999) showed /flag serves an actual HTML form:
 *   <form action="/flag" method="POST">
 *     <input id="answer" name="answer" required type="text" value="">
 *     <input id="val1" name="val1" type="hidden" value="3">
 *     <input id="oper" name="oper" type="hidden" value="+">
 *     <input id="val2" name="val2" type="hidden" value="6">
 *     <input type="submit" value="Get Flag">
 *   </form>
 * POSTing just "answer=9" (the earlier version's guess) got rejected as
 * "weird values / malformed" -- the server expects the hidden val1/oper/val2
 * fields echoed straight back too, not just the computed answer alone. So
 * this version parses the actual hidden field values straight out of the
 * form HTML and POSTs all of them together with the computed answer.
 */
public class MainActivity extends Activity {
    private static final String TAG = "MOBISEC";
    private static final String HOST = "10.0.2.2";
    private static final int PORT = 31337;
    private static final int MAX_FOLLOW_DEPTH = 4;

    // matches "12 + 47", "12+47", "12 * 47", "12 x 47", "12 - 47", "12 / 47"
    private static final Pattern MATH_PATTERN =
            Pattern.compile("(-?\\d+)\\s*([+\\-*x/X])\\s*(-?\\d+)");
    private static final Pattern HREF_PATTERN =
            Pattern.compile("href=\"([^\"]+)\"");
    private static final Pattern FORM_ACTION_PATTERN =
            Pattern.compile("<form\\s+action=\"([^\"]*)\"\\s+method=\"([^\"]*)\"", Pattern.CASE_INSENSITIVE);
    // matches any <input ... name="X" ... type="hidden" ... value="Y" ...>
    // (or with value before type -- attribute order isn't guaranteed)
    private static final Pattern HIDDEN_INPUT_PATTERN =
            Pattern.compile("<input\\b[^>]*\\bname=\"([^\"]+)\"[^>]*\\btype=\"hidden\"[^>]*\\bvalue=\"([^\"]*)\"" +
                    "|<input\\b[^>]*\\bvalue=\"([^\"]*)\"[^>]*\\btype=\"hidden\"[^>]*\\bname=\"([^\"]+)\"");

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        new Thread(this::run).start();
    }

    private void run() {
        try {
            String base = "http://" + HOST + ":" + PORT + "/";
            visit(base, 0);
        } catch (Exception e) {
```

```
        Log.e(TAG, "reachingout failed", e);
    }
}
/** GET a URL, log it, and decide what to do with the response: report the
 * flag if present, solve-and-submit if it's a math question, or follow a
 * link if the page just points somewhere else -- whichever applies. */
private void visit(String url, int depth) {
    if (depth > MAX_FOLLOW_DEPTH) {
        Log.i(TAG, "max follow depth reached at " + url + "; stopping");
        return;
    }
    String resp;
    try {
        resp = httpGet(url);
    } catch (Exception e) {
        Log.i(TAG, "GET " + url + " failed: " + e);
        return;
    }
    Log.i(TAG, "GET " + url + " response: " + resp);
    if (logIfFlag(resp)) return;
    Matcher m = MATH_PATTERN.matcher(resp);
    if (m.find()) {
        long a = Long.parseLong(m.group(1));
        String op = m.group(2);
        long b = Long.parseLong(m.group(3));
        long answer = compute(a, op, b);
        Log.i(TAG, "parsed expression at " + url + ": " + a + " " + op + " " + b + " = " + answer);
        // Prefer submitting exactly what the page's own <form> asks for:
        // its action/method, plus every hidden field echoed back
        // verbatim alongside our computed answer. This is what the
        // server actually validates against (confirmed by #81999
        // rejecting a bare "answer=9" as malformed).
        Matcher fm = FORM_ACTION_PATTERN.matcher(resp);
        if (fm.find()) {
            String action = fm.group(1);
            String method = fm.group(2).toUpperCase();
            String targetUrl = resolveUrl(url, action);
            StringBuilder body = new StringBuilder();
            Matcher hm2 = HIDDEN_INPUT_PATTERN.matcher(resp);
            while (hm2.find()) {
                String name = hm2.group(1) != null ? hm2.group(1) : hm2.group(4);
                String value = hm2.group(1) != null ? hm2.group(2) : hm2.group(3);
                if (body.length() > 0) body.append('&');
                body.append(urlEncode(name)).append('=').append(urlEncode(value));
            }
            if (body.length() > 0) body.append('&');
            body.append("answer=").append(answer);
            Log.i(TAG, "submitting form at " + targetUrl + " via " + method + " with body: " + body);
            if ("POST".equals(method)) {
                tryPost(targetUrl, body.toString());
            } else {
                tryGet(targetUrl + "?" + body);
            }
        }
        // Also try the common bare-answer conventions as a fallback, in
        // case a different page on a later hop doesn't have a <form>.
        tryGet(url + (url.contains("?") ? "&" : "?") + "answer=" + answer);
        tryPost(url, "answer=" + answer);
        return;
    }
```

```
    Matcher hm = HREF_PATTERN.matcher(resp);
    if (hm.find()) {
        String next = hm.group(1);
        Log.i(TAG, "following link found at " + url + " -> " + next);
        visit(next, depth + 1);
        return;
    }
    Log.i(TAG, "no flag, math expression, or link found at " + url + "; nothing more to try");
}
/** Resolves a possibly-relative form action ("/flag") against the page
 * URL it was found on, into a full absolute URL. */
private String resolveUrl(String pageUrl, String action) {
    if (action.startsWith("http://") || action.startsWith("https://")) return action;
    if (action.startsWith("/")) {
        return "http://" + HOST + ":" + PORT + action;
    }
    int lastSlash = pageUrl.lastIndexOf('/');
    return pageUrl.substring(0, lastSlash + 1) + action;
}
private String urlEncode(String s) {
    try {
        return java.net.URLEncoder.encode(s, "UTF-8");
    } catch (Exception e) {
        return s;
    }
}
private long compute(long a, String op, long b) {
    switch (op) {
        case "+": return a + b;
        case "-": return a - b;
        case "*": case "x": case "X": return a * b;
        case "/": return b != 0 ? a / b : 0;
        default: return 0;
    }
}
private void tryGet(String url) {
    try {
        String resp = httpGet(url);
        Log.i(TAG, "GET " + url + " -> " + resp);
        logIfFlag(resp);
    } catch (Exception e) {
        Log.i(TAG, "GET " + url + " failed: " + e);
    }
}
private void tryPost(String url, String body) {
    try {
        HttpURLConnection conn = (HttpURLConnection) new URL(url).openConnection();
        conn.setRequestMethod("POST");
        conn.setDoOutput(true);
        conn.setRequestProperty("Content-Type", "application/x-www-form-urlencoded");
        try (OutputStream os = conn.getOutputStream()) {
            os.write(body.getBytes("UTF-8"));
        }
        String resp = readAll(conn);
        Log.i(TAG, "POST " + url + " (" + body + ") -> " + resp);
        logIfFlag(resp);
    } catch (Exception e) {
        Log.i(TAG, "POST " + url + " failed: " + e);
    }
}
```

```
    private String httpGet(String url) throws Exception {
        HttpURLConnection conn = (HttpURLConnection) new URL(url).openConnection();
        conn.setRequestMethod("GET");
        return readAll(conn);
    }
    private String readAll(HttpURLConnection conn) throws Exception {
        BufferedReader br = new BufferedReader(new InputStreamReader(conn.getInputStream()));
        StringBuilder sb = new StringBuilder();
        String line;
        while ((line = br.readLine()) != null) sb.append(line).append('\n');
        br.close();
        return sb.toString().trim();
    }
    /** Logs the flag distinctly if present in the response. Returns true if
     * a flag was found, so the caller can stop following links/math once
     * the actual goal is reached. */
    private boolean logIfFlag(String resp) {
        if (resp != null && resp.contains("MOBISEC{")) {
            Log.i(TAG, "FLAG: " + resp.substring(resp.indexOf("MOBISEC{")));
            return true;
        }
        return false;
    }
}
```

Notes

Three real submissions drove this to a solve: #81997 showed the root page is only a link to /flag, #81999 showed /flag's real HTML form plus the exact rejection message from a bare answer=9 POST ("You passed some weird values or your HTTP query was malformed"), and #82002 confirmed that echoing the form's hidden val1/oper/val2 fields back alongside the computed answer is accepted.